

Backend · Docker · CI/CD

1) ¿Cómo se relacionan frontend, backend y base de datos dentro de una arquitectura web moderna?

Explica el rol de cada uno y cómo se comunican entre sí.

Pues la BBDD almacena los datos y la estructura de los datos, el backend comunica con base de datos y con front, almacena la lógica en sí de qué se hace con esos datos etc y el front lo muestra en forma de contenedores o lo que toque

2) ¿Qué ventajas ofrece una API REST frente a otros modelos de comunicación entre sistemas?

Menciona al menos dos ventajas desde el punto de vista de escalabilidad o mantenimiento.

Escalabilidad (nuevos servicios) e independencia de sistema operativo/configuración (dockerizable)

3) ¿Qué implica diseñar un backend desacoplado del frontend?

Explica por qué es importante en proyectos grandes o a largo plazo.

Implica separar por completo el servidor de datos y la interfaz del usuario, estableciendo una comunicación mediante una API. Por escalabilidad sin colapsar por dependencias internas.

4) ¿Qué diferencias conceptuales existen entre contenedores Docker y máquinas virtuales?

Indica al menos dos diferencias claras a nivel de recursos o arquitectura.

Los contenedores Docker albergan el kernel completo digamos (configuraciones, sistema operativo, ...) mientras que la máquina virtual solo emula un hardware y tiene sus recursos fijos.

5) ¿Qué significa que una aplicación sea *portable* y cómo ayuda Docker a conseguirlo?

Explica por qué esto es clave en entornos profesionales.

Significa que puede ejecutarse sin cambiar nada de configuración, porque ya viene digamos empaquetado y Docker sirve exactamente para aislar código y empaquetarlo con todas sus dependencias.

6) ¿Qué es la integración continua (CI) y qué problemas evita en un equipo de desarrollo?

Relaciona la respuesta con errores de código o coordinación.

Es la práctica de automatizar la unión y prueba de todo el código para detectar fallos instantáneo. Esto evita el riesgo de integrar todo al final y ver que nada cuadra.

7) ¿Qué es el despliegue continuo (CD) y qué requisitos debería cumplir un proyecto para poder usarlo con seguridad?

Habla de estabilidad, tests o control de versiones.

Es la automatización que lanza cada cambio aprobado a producción. Esto existe una base de código estable gestionado por un estricto control de versiones. Además se utilizan test automatizados a modo “filtro de calidad” para que nada se rompa en vivo.

8) ¿Por qué la automatización es un elemento clave en DevOps?

Explica cómo afecta al tiempo, a la calidad del software y al equipo.

Elimina las tareas repetitivas para reducir tiempos y minimizar errores humanos.

9) ¿Qué diferencias existen entre un error detectado en fase de desarrollo y uno detectado en producción?

Explica las consecuencias técnicas y organizativas.

Uno en desarrollo requiere una corrección inmediata en el código local y otro en producción implica un parche y parado de emergencia, posible pérdida de datos y reputación, y muchas veces, un alto coste económico también.

10) ¿Por qué es importante separar la configuración del código en un backend?

Da ejemplos conceptuales (entornos, seguridad, mantenimiento).

Permite adaptar la app a distintos entornos y proteger secretos (claves API, credenciales bbdd, ...)